UNIVERSIDAD POLITÉCNICA SALESIANA
SEDE QUITO – CAMPUS SUR
CARRERA DE COMPUTACIÓN

INFORME TÉCNICO

Game Pin Pon

Implementación de un juego Pong multijugador mediante sockets UDP con arquitectura cliente-servidor

Proyecto:	Game Pin Pon
Tipo de documento:	Informe técnico en formato IEEE
Tecnologías:	Python 3.13, Pygame 2.6.1, sockets UDP, JSON
Autores:	Josué Samuel Vele y Juan David Figueroa
Repositorio:	github.com/JosueVele77/game_pin_pon
Fecha:	Quito, 23 de octubre de 2026

Quito – Ecuador
2026

CONTENIDO

I. INTRODUCCIÓN 4
A. Objetivo general 4
B. Objetivos específicos 4
C. Alcance 5
II. MARCO TEÓRICO 5
A. Modelo cliente-servidor 5
B. Servidor autoritativo 5
C. Protocolo UDP 5
D. Sockets en Python 5
E. Serialización con JSON 5
F. Pygame 5
III. ARQUITECTURA DEL SISTEMA 5
A. Decisiones de diseño 6
IV. IMPLEMENTACIÓN 7
A. Modelo del juego (modelo.py) 7
B. Servidor (servidor.py) 7
C. Cliente (cliente.py) 8
D. Modo local (juego.py) 9
V. PROTOCOLO DE COMUNICACIÓN 9
A. Mensaje de entrada 9
B. Mensaje de estado 9
C. Mensajes de salida y de servidor lleno 9
D. Tolerancia a las fallas de UDP 10
VI. REGISTROS (LOGS) DEL SERVIDOR 10
VII. INSTALACIÓN Y USO 11
A. Requisitos 11
B. Instalación 11
C. Puesta en marcha 11
D. Controles 12
E. Opciones de línea de comandos 12
VIII. RESULTADOS Y EVIDENCIAS 12
IX. PRUEBAS REALIZADAS 14
X. SOLUCIÓN DE PROBLEMAS 14
A. «Sin respuesta. Revisa IP, puerto, red y cortafuegos.» 14
B. «Servidor lleno: ya hay dos jugadores.» 15
C. «Falta Pygame» 15
XI. LIMITACIONES Y TRABAJO FUTURO 15
XII. CONCLUSIONES 15

REFERENCIAS 15
ANEXO A. CÓDIGO FUENTE COMPLETO 16
A-1. modelo.py 16
A-2. servidor.py 18
A-3. cliente.py 20
A-4. juego.py 25

Game Pin Pon: implementación de un juego Pong multijugador mediante sockets UDP con arquitectura cliente-servidor

Josué Samuel Vele
Carrera de Computación, Universidad Politécnica Salesiana
Sede Quito – Campus Sur, Quito, Ecuador

Resumen*—Este documento describe el diseño, la implementación y la validación de Game Pin Pon, una versión del clásico juego Pong que permite a dos jugadores competir desde computadoras distintas de una misma red. El sistema adopta una arquitectura cliente-servidor sobre el protocolo UDP, en la cual un servidor autoritativo ejecuta la física y las reglas a 60 actualizaciones por segundo y difunde el estado de la partida a los clientes, que únicamente envían sus controles y dibujan lo recibido mediante Pygame. Se detallan el modelo del juego, el protocolo de mensajes en formato JSON, los mecanismos para tolerar la pérdida, duplicación y desorden de datagramas, los registros del servidor y las pruebas realizadas, junto con evidencias de ejecución y el código fuente completo.*

Palabras clave*—Pong, sockets, UDP, cliente-servidor, servidor autoritativo, Pygame, juegos multijugador, Python.*

Abstract*—This paper presents the design, implementation and validation of Game Pin Pon, a networked version of the classic Pong game for two players on the same network. The system follows a client-server architecture over UDP: an authoritative server runs the physics and game rules at 60 updates per second and broadcasts the game state, while clients only send their inputs and render the received state using Pygame. The game model, the JSON message protocol, the mechanisms used to tolerate lost, duplicated and out-of-order datagrams, the server logs and the tests performed are described, together with execution evidence and the full source code.*

Index Terms*—Pong, sockets, UDP, client-server, authoritative server, Pygame, multiplayer games, Python.*

I. INTRODUCCIÓN

Los juegos en red son un excelente caso de estudio para la programación con sockets, ya que exigen intercambiar datos con baja latencia, mantener sincronizados a varios participantes y tolerar las imperfecciones de la red. Pong, por su simplicidad —dos paletas, una pelota y un marcador— permite concentrarse en estos aspectos sin que la complejidad de las reglas oculte los conceptos de comunicación.

Game Pin Pon es una implementación de Pong escrita en Python que separa con claridad tres responsabilidades: las **reglas del juego** (`modelo.py`), la **comunicación en red** (`servidor.py` y la clase `ConexionCliente` de `cliente.py`) y la **presentación gráfica** (`PantallaPong`, basada en Pygame). Además de la modalidad en red, el proyecto incluye un modo local para dos jugadores en un mismo teclado (`juego.py`), que reutiliza las mismas reglas.

A. Objetivo general

Desarrollar un juego Pong multijugador en red que permita a dos jugadores competir desde equipos distintos mediante sockets UDP, con un servidor que controle de forma autoritativa el estado de la partida.

B. Objetivos específicos

- Diseñar un modelo del juego independiente de la red y de la interfaz gráfica.
- Definir un protocolo de mensajes JSON sobre UDP para enviar controles y difundir el estado.
- Incorporar mecanismos de robustez frente a pérdida, duplicación y desorden de paquetes.
- Registrar en consola los eventos relevantes del servidor (conexiones y desconexiones).

- Validar el funcionamiento con dos computadoras reales y documentar las evidencias.

C. Alcance

El sistema admite **dos jugadores simultáneos** por servidor. La partida finaliza cuando uno de ellos alcanza 10 puntos. El proyecto está pensado para redes de confianza (red local o institucional): no incluye cifrado ni autenticación.

II. MARCO TEÓRICO

A. Modelo cliente-servidor

En el modelo cliente-servidor, un proceso servidor ofrece un servicio y uno o varios clientes lo consumen [5]. En juegos multijugador existen dos variantes básicas: la comunicación entre pares y el servidor central. En esta última, los clientes no calculan el resultado de sus acciones; envían sus entradas y reciben el estado resultante, lo que evita trampas y desincronizaciones [6].

B. Servidor autoritativo

Un servidor autoritativo es la única fuente de verdad sobre el estado del juego. Los clientes actúan como terminales «ligeras»: capturan el teclado, envían las teclas pulsadas y dibujan el último estado recibido. En Game Pin Pon, la posición de la pelota, las paletas, los puntos y el ganador se calculan exclusivamente en el servidor.

C. Protocolo UDP

El Protocolo de Datagramas de Usuario (UDP) ofrece un servicio sin conexión, sin confirmación de entrega, sin retransmisiones y sin garantía de orden [1]. A cambio, introduce muy poca sobrecarga y latencia, características deseables en aplicaciones en tiempo real donde un dato antiguo pierde valor rápidamente. Como contrapartida, la aplicación debe tolerar que un paquete se pierda, se duplique o llegue desordenado; en este proyecto se resuelve con números de secuencia, reenvío continuo de las entradas y contadores acumulativos.

D. Sockets en Python

El módulo `socket` de la biblioteca estándar de Python expone la interfaz de sockets del sistema operativo [2]. El proyecto utiliza sockets `AF_INET` de tipo `SOCK_DGRAM` en modo **no bloqueante**, de forma que ni el bucle del servidor ni el dibujado del cliente se detengan esperando datos.

E. Serialización con JSON

Los mensajes se serializan en JSON [4] y se codifican en UTF-8. Aunque es menos compacto que un formato binario, es legible, fácil de depurar y se valida de forma sencilla en el receptor.

F. Pygame

Pygame es una biblioteca multimedia para Python basada en SDL [3]. Se emplea para crear la ventana, capturar eventos y teclado, y dibujar paletas, pelota y textos a 60 cuadros por segundo.

III. ARQUITECTURA DEL SISTEMA

El sistema se compone de un servidor y de dos clientes. El servidor recibe los controles de ambos jugadores, avanza la simulación con ayuda del modelo y difunde el estado resultante. Cada cliente envía un paquete de entrada por cuadro y dibuja el estado más reciente que recibe.

```
Cliente 1 (Jugador 1)                Servidor                    Cliente 2 (Jugador 2)
  Pygame + UDP          entrada      modelo.Partida      entrada     Pygame + UDP
  paleta izquierda  ---------------> (física y reglas) <------------ paleta derecha
                    <---------------   60 ciclos/s     ------------->
                        estado                             estado
```

La TABLA I resume los archivos del proyecto y su responsabilidad; la estructura real de la carpeta se muestra en la Fig. 1.

TABLA I
MÓDULOS DEL PROYECTO Y SU RESPONSABILIDAD

Archivo	Responsabilidad
`modelo.py`	Reglas del juego: paletas, pelota, colisiones, puntos, pausa y ganador. No conoce sockets ni Pygame.
`servidor.py`	Servidor UDP autoritativo: registra jugadores, aplica controles, avanza la partida y difunde el estado.
`cliente.py`	Cliente de red (`ConexionCliente`), validación de paquetes e interfaz gráfica (`PantallaPong`, `ClientePong`). Puede distribuirse como archivo único.
`juego.py`	Modo local para dos jugadores en un mismo teclado, reutilizando `modelo.py` y `PantallaPong`.

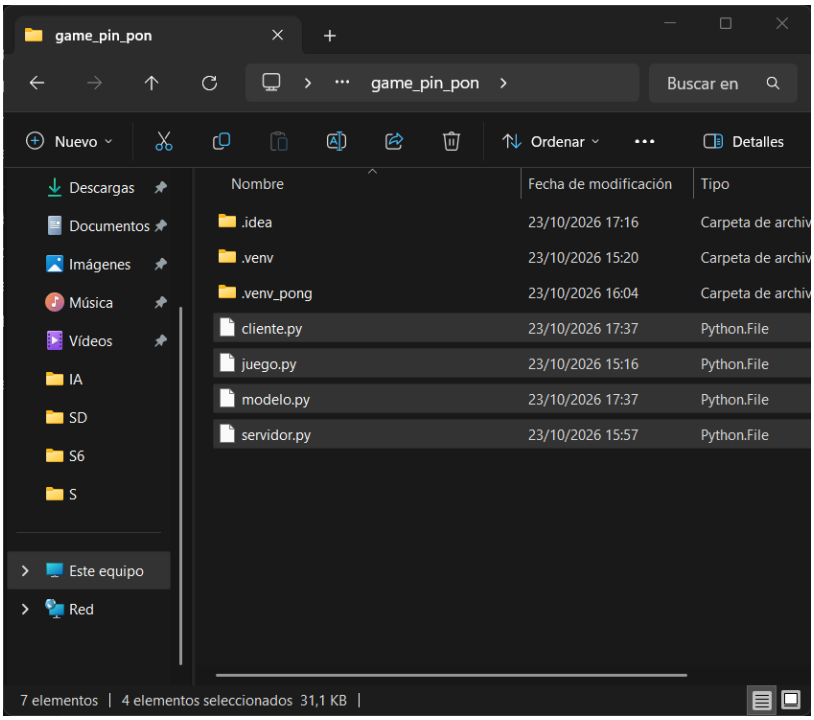


Fig. 1. Estructura de archivos del proyecto Game Pin Pon.

A. Decisiones de diseño

- **Separación de responsabilidades:** el modelo se prueba y reutiliza sin red ni ventana.
- **Cliente autónomo:** `cliente.py` replica las constantes de pantalla para poder entregarse al segundo jugador sin dependencias del resto del proyecto.
- **Importación diferida de Pygame:** permite probar la lógica de red en equipos sin Pygame instalado.

- **Sockets no bloqueantes:** el servidor mantiene su ritmo de 60 ciclos por segundo aunque no lleguen datos.

IV. IMPLEMENTACIÓN

A. Modelo del juego (modelo.py)

Las posiciones se expresan en píxeles y las velocidades en píxeles por segundo. La TABLA II reúne las constantes del modelo.

TABLA II
CONSTANTES DEL MODELO DEL JUEGO

Constante	Valor	Significado
`ANCHO_PANTALLA` × `ALTO_PANTALLA`	800 × 600	Dimensiones de la pista (px)
`ANCHO_PALETA` × `ALTO_PALETA`	14 × 90	Dimensiones de cada paleta (px)
`MARGEN_PALETA`	25 px	Distancia de la paleta al borde lateral
`RADIO_PELOTA`	12 px	Radio de la pelota
`VELOCIDAD_PALETA`	480 px/s	Velocidad de desplazamiento vertical
`PUNTOS_PARA_GANAR`	10	Puntos necesarios para ganar

El modelo contiene tres clases:

- **`Paleta`:** guarda su posición y limita el movimiento al interior de la pista, con un margen de 10 px arriba y abajo. `mover(direccion, segundos)` recibe −1 (subir), 1 (bajar) o 0 (quieta).
- **`Pelota`:** almacena posición y velocidad (inicialmente 380 px/s en X y 280 px/s en Y). Rebota en los bordes superior e inferior y en las paletas. Tras cada rebote en paleta, la velocidad horizontal se multiplica por 1,08 (máximo 950 px/s) y la vertical por 1,05 (máximo ±750 px/s), de modo que el juego se acelera progresivamente. Después del rebote la pelota se coloca fuera de la paleta para evitar rebotes repetidos.
- **`Partida`:** compone dos paletas y una pelota, administra puntos, ganador, pausa y reinicio, y expone `obtener_estado()`, que devuelve un diccionario simple apto para dibujarse o enviarse como JSON.

Para evitar que la pelota atraviese una paleta cuando va rápido, `Partida.actualizar` divide el recorrido de cada cuadro en tramos de aproximadamente 6 px y comprueba bordes, paletas y puntuación en cada tramo. Cuando la pelota sale por un lado, se anota un punto para el jugador contrario y la pelota vuelve al centro con una dirección aleatoria.

B. Servidor (servidor.py)

`ServidorPong` crea un socket UDP no bloqueante enlazado a la dirección y puerto indicados (por defecto `0.0.0.0:12345`) y mantiene una lista de dos puestos de jugador. La TABLA III muestra sus parámetros.

TABLA III
CONSTANTES DEL SERVIDOR

Constante	Valor	Uso
`PUERTO_PREDETERMINADO`	12345	Puerto UDP de escucha
`TIEMPO_DESCONEXION`	5,0 s	Inactividad tras la cual se libera el puesto
`VIGENCIA_CONTROLES`	0,3 s	Si no llegan controles, la paleta se detiene

Constante	Valor	Uso
`ACTUALIZACIONES_POR_SEGUNDO`	60	Frecuencia del bucle principal
`MAXIMO_MENSAJES_POR_CICLO`	256	Tope de paquetes leídos por ciclo

El bucle principal (`ejecutar`) repite, aproximadamente 60 veces por segundo, cuatro pasos:

1. **Recepción:** se retiran los jugadores inactivos y se leen hasta 256 datagramas pendientes; los que no son JSON válido se ignoran.
2. **Actualización:** la partida avanza únicamente si hay **dos** jugadores conectados; el intervalo de tiempo se limita a 0,05 s para evitar saltos grandes.
3. **Difusión:** se incrementa el número de secuencia y se envía el estado a cada jugador, añadiendo su `numero_jugador`.
4. **Espera:** se duerme el tiempo restante hasta completar 1/60 s.

Registro de jugadores. El primer mensaje `entrada` de una dirección desconocida la asigna al primer puesto libre: el Jugador 1 controla la paleta izquierda y el Jugador 2 la derecha. Si ambos puestos están ocupados se responde `{"tipo": "lleno"}`. Al recibir `salir` o tras 5 s sin actividad se libera el puesto y se **reinicia la partida**.

Pausa sincronizada. El cliente envía un contador acumulado `solicitud_pausa`. El servidor aplica la diferencia respecto al último valor recibido y solo alterna la pausa si esa diferencia es impar, y únicamente cuando hay dos jugadores y no existe un ganador. Así, un datagrama duplicado no activa la pausa dos veces y la pérdida de un paquete no pierde la pulsación, porque el contador se reenvía en cada entrada.

Controles con vigencia. Cada jugador conserva sus últimas teclas (`subir`, `bajar`). Si pasan 0,3 s sin recibir entradas, su paleta se detiene, lo que evita movimientos «fantasma» cuando se interrumpe la comunicación.

Opción `--jugar`. Al iniciar el servidor con `--jugar`, se lanza un subproceso con `cliente.py` conectado a `127.0.0.1`, de modo que el anfitrión juega como Jugador 1 sin abrir otra terminal.

C. Cliente (cliente.py)

El cliente se organiza en cuatro piezas:

- **`validar_estado`:** rechaza paquetes incompletos o con valores que no se pueden dibujar (tipos incorrectos, números no finitos, puntos fuera de 0–10, jugador fuera de {0, 1}, etc.).
- **`ConexionCliente`:** abre un socket UDP asociado al servidor con `connect()` (que en UDP solo fija el destino y filtra respuestas de otras direcciones), envía un paquete `entrada` por cuadro y lee las respuestas pendientes. Solo acepta un estado si su `secuencia` es mayor que la última recibida, descartando paquetes antiguos o desordenados.
- **`PantallaPong`:** dibuja la pista, las paletas, la pelota, el marcador y los textos; no conoce sockets ni modifica la partida.
- **`ClientePong`:** coordina el formulario de conexión, el teclado, la red y el dibujo siguiendo el orden: eventos → intercambio de red → dibujo → espera del reloj.

Los estados de conexión se comunican al usuario con mensajes en pantalla: «Conectando al servidor…», «Esperando al otro jugador… (1/2)», «Se perdió la conexión…» (más de 1 s sin estado) y, tras 5 s sin respuesta, «Sin respuesta. Revisa IP, puerto, red y cortafuegos.» Las teclas solo se envían si la ventana tiene el foco.

D. Modo local (juego.py)

`JuegoLocal` compone una `Partida` y una `PantallaPong` en un mismo equipo. El jugador izquierdo usa `W`/`S` y el derecho las flechas ↑/↓; `P` pausa y `R` reinicia. Limita el salto temporal a 0,05 s para evitar avances bruscos al arrastrar la ventana.

V. PROTOCOLO DE COMUNICACIÓN

Todos los mensajes son objetos JSON codificados en UTF-8 y enviados como datagramas UDP. La TABLA IV resume los tipos de mensaje.

TABLA IV
TIPOS DE MENSAJE DEL PROTOCOLO

Tipo	Sentido	Descripción
`entrada`	Cliente → Servidor	Teclas pulsadas, solicitud de reinicio y contador de pausa; se envía cada cuadro (~60/s).
`salir`	Cliente → Servidor	Aviso de desconexión voluntaria.
`estado`	Servidor → Cliente	Estado completo de la partida con número de secuencia (~60/s).
`lleno`	Servidor → Cliente	Respuesta cuando ya hay dos jugadores registrados.

A. Mensaje de entrada

```
{
  "tipo": "entrada",
  "subir": true,
  "bajar": false,
  "reiniciar": false,
  "solicitud_pausa": 3
}
```

B. Mensaje de estado

```
{
  "tipo": "estado",
  "secuencia": 1042,
  "numero_jugador": 0,
  "jugadores_conectados": 2,
  "posicion_pelota": [400.0, 300.0],
  "paleta_izquierda": 255.0,
  "paleta_derecha": 255.0,
  "puntos_izquierda": 1,
  "puntos_derecha": 3,
  "ganador": null,
  "en_pausa": false
}
```

C. Mensajes de salida y de servidor lleno

```
{ "tipo": "salir" }
{ "tipo": "lleno" }
```

D. Tolerancia a las fallas de UDP

La TABLA V relaciona cada problema típico de UDP con el mecanismo que lo mitiga.

TABLA V
MECANISMOS FRENTE A LAS LIMITACIONES DE UDP

Problema	Mecanismo en Game Pin Pon
Paquetes fuera de orden	Número de `secuencia` en cada estado; el cliente descarta los menores o iguales al último recibido.
Paquetes perdidos	El estado y las entradas se reenvían de forma continua a 60 Hz; un paquete perdido es reemplazado por el siguiente.
Paquetes duplicados	El contador `solicitud_pausa` es acumulativo; el servidor solo aplica la diferencia con el último valor visto.
Datos corruptos o malformados	El servidor ignora JSON inválido y el cliente valida cada campo con `validar_estado`.
Jugador que deja de responder	Tras 0,3 s sin controles la paleta se detiene; tras 5 s se libera el puesto.
Servidor inalcanzable	El cliente muestra un error tras 5 s sin respuesta.

VI. REGISTROS (LOGS) DEL SERVIDOR

El servidor informa por consola los eventos más importantes, con el prefijo `[SERVIDOR]`. Estos registros permiten comprobar que el servicio está activo, conocer la dirección que debe usar el otro jugador y verificar quién se conectó. La TABLA VI lista los mensajes.

TABLA VI
MENSAJES DE REGISTRO DEL SERVIDOR

Mensaje	Cuándo aparece
`Escuchando en 0.0.0.0:12345 (UDP).`	Al iniciar el servidor correctamente.
`Direcciones de esta PC: <IPv4>`	Después de iniciar; muestra las IPv4 disponibles (se excluyen las 127.x).
`Jugador N: ('IP', puerto)`	Cuando un cliente se registra en el puesto N.
`Jugador N desconectado.`	Al recibir `salir` o por inactividad superior a 5 s.
`Cerrado.`	Al detener el servidor con Ctrl+C.

La Fig. 2 muestra una ejecución real con `python servidor.py --jugar`: el servidor escucha en el puerto 12345 y su IP es 172.17.212.166. El **Jugador 1** corresponde a `127.0.0.1`, es decir, el cliente local lanzado por `--jugar`, y el **Jugador 2** es la dirección 172.17.211.222, una segunda computadora conectada por red.



Fig. 2. Registros del servidor con dos jugadores conectados.

VII. INSTALACIÓN Y USO

A. Requisitos

- Python 3.10 o superior (probado con Python 3.13.14).
- Pygame (probado con la versión 2.6.1), solo necesario en los equipos que ejecutan un cliente.
- Dos computadoras en la misma red, o una sola usando la dirección `127.0.0.1`.

B. Instalación

```
python -m venv .venv
# Windows (PowerShell)
.\.venv\Scripts\Activate.ps1
# Linux / macOS
source .venv/bin/activate

python -m pip install pygame
```

C. Puesta en marcha

1. **Iniciar el servidor** en el equipo anfitrión. Con `--jugar`, además abre su propio cliente como Jugador 1:

```
python servidor.py --jugar
```

2. **Conocer la IP** del servidor: se imprime en consola (o con `ipconfig` en Windows).
3. **Conectar el segundo jugador.** En otro equipo, copiar únicamente `cliente.py` y ejecutar:

```
python cliente.py --direccion 172.17.212.166 --puerto 12345
```

4. **Jugar.** Cuando ambos jugadores están conectados la partida inicia sola. Gana quien llegue primero a 10 puntos.

También se puede ejecutar `python cliente.py` sin argumentos y escribir la IP y el puerto en el formulario (Tab cambia de campo, Intro conecta). El modo local se inicia con `python juego.py`.

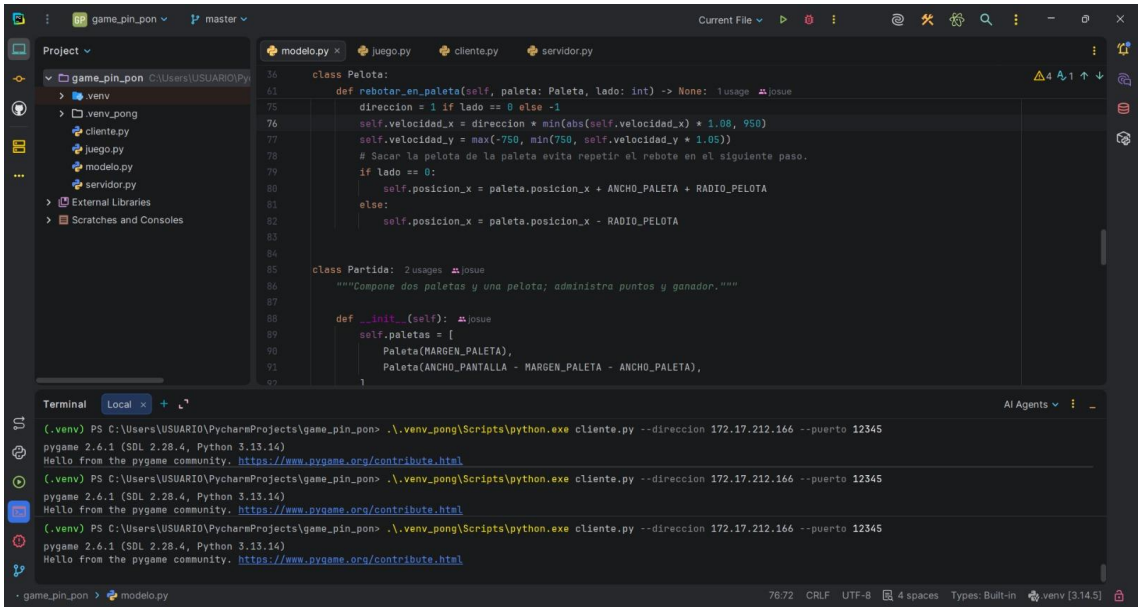


Fig. 3. Ejecución del cliente desde la terminal del segundo equipo, con el intérprete del entorno virtual.

D. Controles

TABLA VII
CONTROLES DEL MODO EN RED

Tecla	Acción
`W` / `↑`	Subir la paleta
`S` / `↓`	Bajar la paleta
`P`	Pausar o reanudar la partida (afecta a ambos jugadores)
`R`	Iniciar una nueva partida cuando hay un ganador
`Esc`	Desconectarse y volver al formulario

En el modo local, el jugador izquierdo usa `W`/`S`, el derecho `↑`/`↓`, `P` pausa y `R` reinicia.

E. Opciones de línea de comandos

TABLA VIII
ARGUMENTOS DE SERVIDOR.PY Y CLIENTE.PY

Programa	Opción	Descripción	Por defecto
`servidor.py`	`--direccion`	Interfaz de escucha	`0.0.0.0`
`servidor.py`	`--puerto`	Puerto UDP	`12345`
`servidor.py`	`--jugar`	Abre un cliente local conectado al servidor	desactivado
`cliente.py`	`--direccion`	IP o nombre del servidor	vacío (formulario)
`cliente.py`	`--puerto`	Puerto UDP del servidor	`12345`

VIII. RESULTADOS Y EVIDENCIAS

Se probó el sistema con dos computadoras: una ejecutó el servidor junto con el cliente local (Jugador 1) y otra se conectó por red como Jugador 2. Cada ventana indica «Tu paleta: izquierda» o «Tu paleta: derecha» según el `numero_jugador` asignado por el servidor, y el marcador se muestra en la parte superior.

Durante las primeras pruebas, el formulario de conexión mostró el mensaje «Sin respuesta. Revisa IP, puerto, red y cortafuegos.» (Fig. 4). Este aviso aparece cuando el cliente no recibe ningún estado en 5 s y orienta al usuario a revisar la configuración de red; sus causas habituales se describen en la sección X.

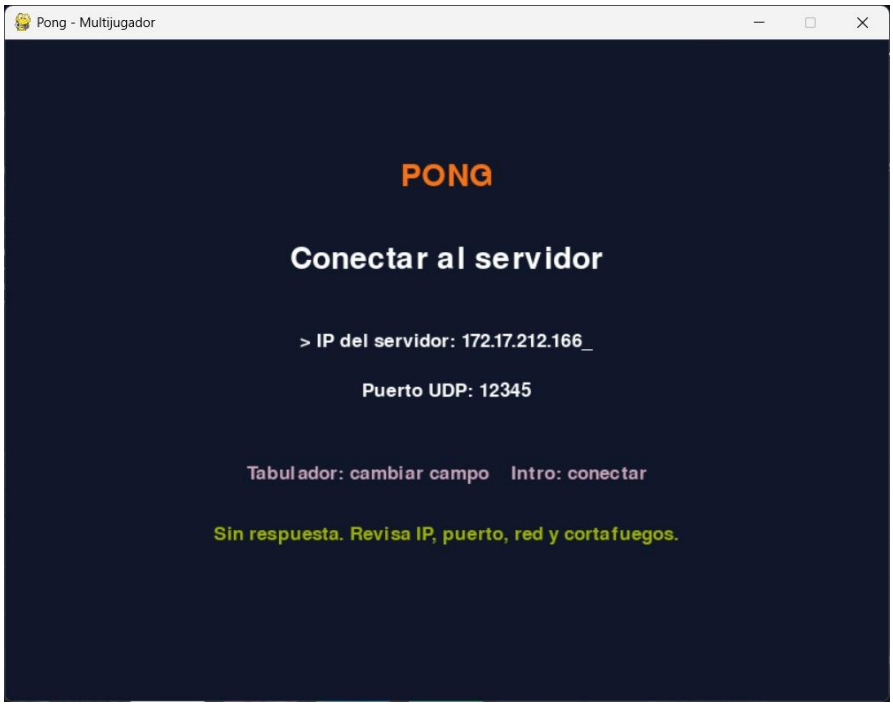


Fig. 4. Formulario de conexión con el aviso «Sin respuesta» tras 5 s sin respuesta del servidor.

Una vez establecida la comunicación, ambos jugadores vieron la misma partida en tiempo real. La Fig. 5 corresponde a la vista del Jugador 1 (paleta izquierda) y la Fig. 6 a la del Jugador 2 (paleta derecha); en cada captura el marcador es el que la partida tenía en ese instante.

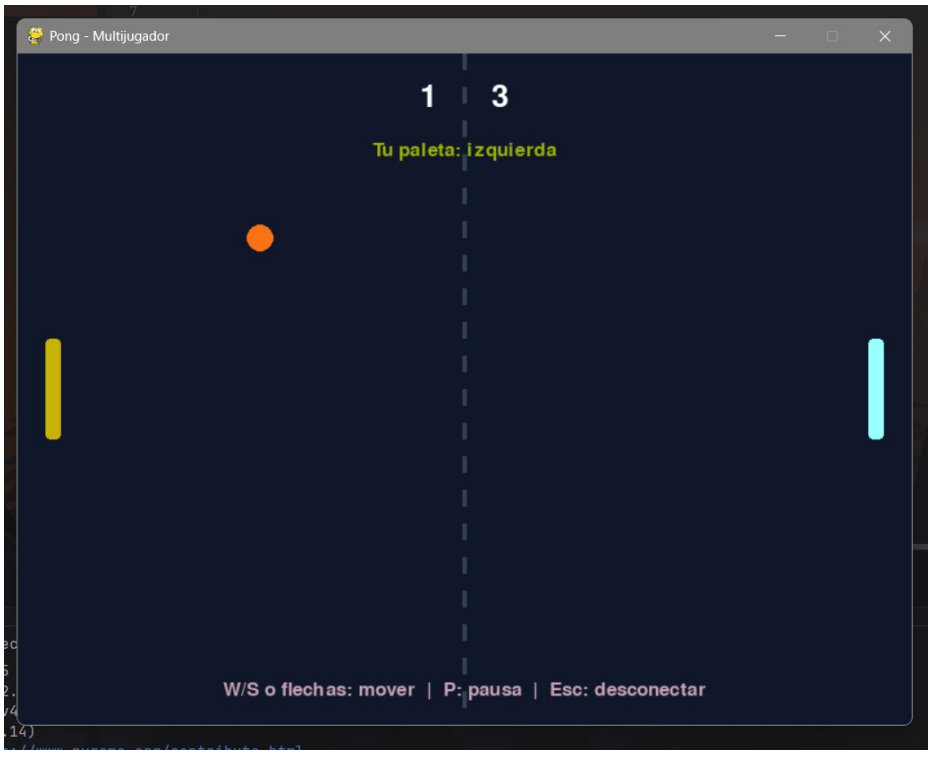


Fig. 5. Vista del Jugador 1 (paleta izquierda) durante una partida en red.

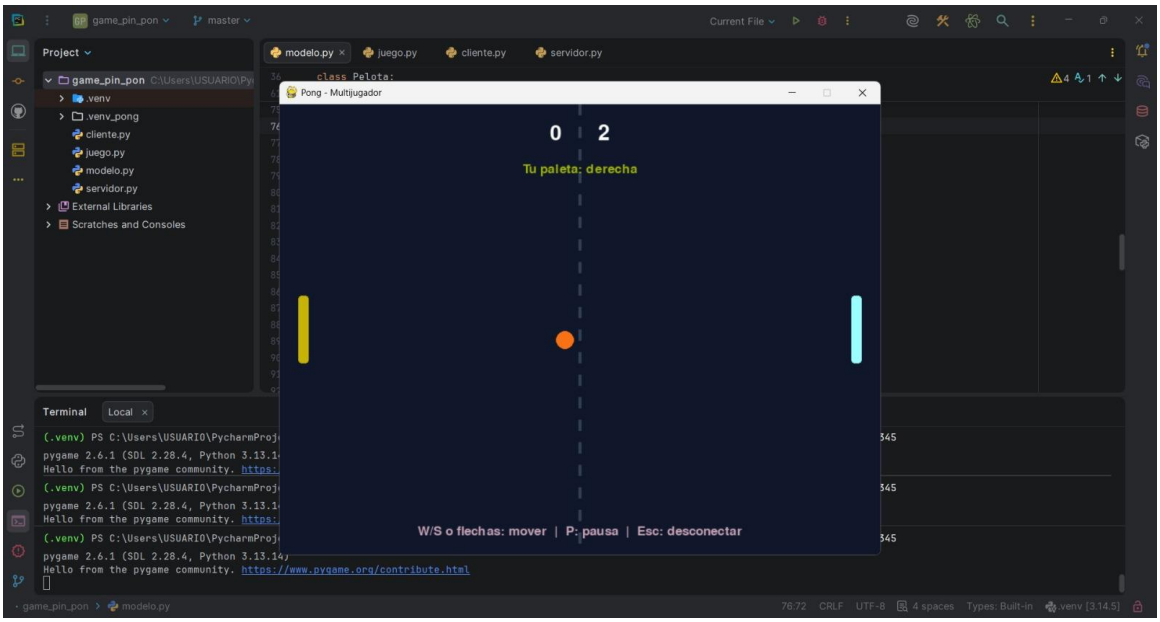


Fig. 6. Vista del Jugador 2 (paleta derecha) durante una partida en red, ejecutada desde el IDE.

IX. PRUEBAS REALIZADAS

TABLA IX
CASOS DE PRUEBA FUNCIONALES

Prueba	Resultado esperado
Servidor y cliente local (`--jugar`)	El Jugador 1 se registra y la pantalla indica «Esperando al otro jugador… (1/2)».
Segundo cliente por red	Se registra como Jugador 2 y la partida comienza automáticamente.
Tercer cliente	Recibe el mensaje «Servidor lleno: ya hay dos jugadores.»
Pausa con `P`	Ambos jugadores ven «JUEGO PAUSADO»; `P` la reanuda para los dos.
Cierre de un cliente con `Esc`	El servidor registra la desconexión y reinicia la partida.
Servidor apagado o inalcanzable	El cliente muestra «Sin respuesta…» a los 5 s.
Llegar a 10 puntos	Se muestra el ganador; `R` inicia una nueva partida.
Paquete con datos inválidos	El servidor ignora el JSON malformado y el cliente descarta estados que no superan `validar_estado`.

X. SOLUCIÓN DE PROBLEMAS

A. «Sin respuesta. Revisa IP, puerto, red y cortafuegos.»

1. **IP correcta:** usar la IPv4 impresa por el servidor (o `ipconfig`).
2. **Misma red:** comprobar la conectividad con `ping <IP-del-servidor>`.
3. **Cortafuegos:** permitir el puerto UDP 12345 de entrada en el equipo servidor. En PowerShell con permisos de administrador:

```
netsh advfirewall firewall add rule name="Pong UDP" dir=in action=allow protocol=UDP localport=12345
```

4. **Servidor activo:** verificar que aparece «Escuchando en 0.0.0.0:12345 (UDP)».
5. **Aislamiento de clientes:** algunas redes institucionales o de invitados bloquean la comunicación entre dispositivos.

B. «Servidor lleno: ya hay dos jugadores.»

El servidor admite solo dos jugadores. Se debe esperar a que uno se desconecte (`Esc`) o quede inactivo durante 5 s.

C. «Falta Pygame»

```
python -m pip install pygame
```

XI. LIMITACIONES Y TRABAJO FUTURO

- **Capacidad de dos jugadores.** El servidor define dos puestos (`self.jugadores = [None, None]`), el modelo tiene dos paletas y el cliente valida `numero_jugador` en {0, 1}. Para admitir hasta cuatro participantes habría que ampliar la lista de jugadores, las paletas y la geometría de la pista en `modelo.py`, definir un marcador por equipos o por jugador y actualizar las validaciones de `cliente.py`.
- **Fiabilidad.** UDP no garantiza la entrega; se mitiga con secuencias y reenvío continuo, pero no hay retransmisión ni predicción en el cliente.
- **Seguridad.** No existe cifrado ni autenticación; se recomienda usarlo en redes de confianza.
- **Mejoras posibles.** Sala de espera con nombres de jugador, sonidos, predicción y reconciliación en el cliente, y pruebas automatizadas del modelo y del protocolo.

XII. CONCLUSIONES

- Se desarrolló un Pong multijugador funcional en el que un servidor autoritativo controla la partida y dos clientes se comunican con él mediante UDP, validado entre dos computadoras reales.
- La separación entre modelo, red y presentación permitió reutilizar las mismas reglas en el servidor y en el modo local, y facilita probar cada parte por separado.
- Los números de secuencia, el reenvío continuo de entradas, el contador acumulativo de pausa y la validación de paquetes bastan para que el juego sea estable pese a las limitaciones de UDP.
- Los registros del servidor y los mensajes de estado del cliente simplifican el diagnóstico de problemas de conexión, que en la práctica se deben con más frecuencia al cortafuegos o a la configuración de la red.
- El diseño actual admite dos jugadores; ampliarlo a más participantes es viable, pero requiere modificar el modelo, el servidor y las validaciones del cliente.

REFERENCIAS

[1] J. Postel, «User Datagram Protocol», IETF, RFC 768, ago. 1980.

[2] Python Software Foundation, «socket — Low-level networking interface», *Python 3 Documentation*. [En línea]. Disponible: https://docs.python.org/3/library/socket.html

[3] Pygame Community, «Pygame Documentation». [En línea]. Disponible: https://www.pygame.org/docs/

[4] T. Bray, Ed., «The JavaScript Object Notation (JSON) Data Interchange Format», IETF, RFC 8259, dic. 2017.

[5] J. F. Kurose y K. W. Ross, *Computer Networking: A Top-Down Approach*, 7.ª ed. Boston, MA, EE. UU.: Pearson, 2016.

[6] G. Gambetta, «Fast-Paced Multiplayer: Client-Server Game Architecture». [En línea]. Disponible: https://www.gabrielgambetta.com/client-server-game-architecture.html

[7] J. Vele, «game_pin_pon», repositorio de GitHub. [En línea]. Disponible: https://github.com/JosueVele77/game_pin_pon

ANEXO A. CÓDIGO FUENTE COMPLETO

Se incluye el código fuente íntegro de los cuatro archivos del proyecto, disponible también en el repositorio https://github.com/JosueVele77/game_pin_pon.

A-1. modelo.py

```
"""Objetos y reglas de Pong, independientes de la red y de la ventana.

Todas las posiciones se expresan en píxeles y las velocidades en píxeles por
segundo. El servidor y el modo local utilizan las mismas reglas.
"""

from dataclasses import dataclass
import random



ANCHO_PANTALLA = 800
ALTO_PANTALLA = 600
ANCHO_PALETA = 14
ALTO_PALETA = 90
MARGEN_PALETA = 25
RADIO_PELOTA = 12
VELOCIDAD_PALETA = 480
PUNTOS_PARA_GANAR = 10



@dataclass
class Paleta:
    """Representa una paleta y limita su movimiento al interior de la pista."""

    posicion_x: float
    posicion_y: float = (ALTO_PANTALLA - ALTO_PALETA) / 2

    def mover(self, direccion: int, segundos: float) -> None:
        """Recibe -1 para subir, 1 para bajar y 0 para permanecer quieta."""
        limite_inferior = ALTO_PANTALLA - ALTO_PALETA - 10
        desplazamiento = direccion * VELOCIDAD_PALETA * segundos
        self.posicion_y = max(10, min(limite_inferior, self.posicion_y + desplazamiento))



@dataclass
class Pelota:
    """Guarda la posición y la velocidad de la pelota."""

    posicion_x: float = ANCHO_PANTALLA / 2
    posicion_y: float = ALTO_PANTALLA / 2
    velocidad_x: float = 380
    velocidad_y: float = 280

    def reiniciar(self) -> None:
        """Coloca la pelota en el centro y elige la dirección del siguiente saque."""
        self.posicion_x = ANCHO_PANTALLA / 2
        self.posicion_y = ALTO_PANTALLA / 2
        self.velocidad_x = random.choice((-380, 380))
        self.velocidad_y = random.choice((-280, 280))

    def mover(self, segundos: float) -> None:
        self.posicion_x += self.velocidad_x * segundos
        self.posicion_y += self.velocidad_y * segundos

    def rebotar_en_bordes(self) -> None:
        limite_inferior = ALTO_PANTALLA - RADIO_PELOTA
        if self.posicion_y < RADIO_PELOTA or self.posicion_y > limite_inferior:
            self.posicion_y = max(RADIO_PELOTA, min(limite_inferior, self.posicion_y))
            self.velocidad_y *= -1

    def rebotar_en_paleta(self, paleta: Paleta, lado: int) -> None:
        """El lado 0 es el izquierdo; el lado 1 es el derecho."""
```

```
        se_acerca = self.velocidad_x < 0 if lado == 0 else self.velocidad_x > 0
        coincide_en_x = (
            self.posicion_x + RADIO_PELOTA >= paleta.posicion_x
            and self.posicion_x - RADIO_PELOTA <= paleta.posicion_x + ANCHO_PALETA
        )
        coincide_en_y = (
            self.posicion_y + RADIO_PELOTA >= paleta.posicion_y
            and self.posicion_y - RADIO_PELOTA <= paleta.posicion_y + ALTO_PALETA
        )
        if not (se_acerca and coincide_en_x and coincide_en_y):
            return


        direccion = 1 if lado == 0 else -1
        self.velocidad_x = direccion * min(abs(self.velocidad_x) * 1.08, 950)
        self.velocidad_y = max(-750, min(750, self.velocidad_y * 1.05))
        # Sacar la pelota de la paleta evita repetir el rebote en el siguiente paso.
        if lado == 0:
            self.posicion_x = paleta.posicion_x + ANCHO_PALETA + RADIO_PELOTA
        else:
            self.posicion_x = paleta.posicion_x - RADIO_PELOTA



class Partida:
    """Compone dos paletas y una pelota; administra puntos y ganador."""

    def __init__(self):
        self.paletas = [
            Paleta(MARGEN_PALETA),
            Paleta(ANCHO_PANTALLA - MARGEN_PALETA - ANCHO_PALETA),
        ]
        self.pelota = Pelota()
        self.reiniciar()


    def reiniciar(self) -> None:
        self.puntos = [0, 0]
        self.ganador = None
        self.en_pausa = False
        for paleta in self.paletas:
            paleta.posicion_y = (ALTO_PANTALLA - ALTO_PALETA) / 2
        self.pelota.reiniciar()


    def actualizar(self, segundos: float, direcciones: tuple) -> None:
        """Avanza una partida activa según el tiempo y las entradas de ambos jugadores."""
        if self.ganador is not None or self.en_pausa:
            return
        for paleta, direccion in zip(self.paletas, direcciones):
            paleta.mover(direccion, segundos)


        # Dividir el recorrido en tramos cortos evita atravesar paletas al acelerar.
        velocidad_maxima = max(abs(self.pelota.velocidad_x), abs(self.pelota.velocidad_y))
        pasos = max(1, int(velocidad_maxima * segundos / 6) + 1)
        for _ in range(pasos):
            self.pelota.mover(segundos / pasos)
            self.pelota.rebotar_en_bordes()
            for lado, paleta in enumerate(self.paletas):
                self.pelota.rebotar_en_paleta(paleta, lado)
            if self.registrar_punto():
                break


    def registrar_punto(self) -> bool:
        """Devuelve True al salir la pelota; anota y prepara el siguiente saque."""
        if self.pelota.posicion_x < -RADIO_PELOTA:
            anotador = 1
        elif self.pelota.posicion_x > ANCHO_PANTALLA + RADIO_PELOTA:
            anotador = 0
        else:
            return False
        self.puntos[anotador] += 1
        if self.puntos[anotador] >= PUNTOS_PARA_GANAR:
            self.ganador = anotador
        self.pelota.reiniciar()
        return True


    def obtener_estado(self) -> dict:
        """Produce datos simples que pueden dibujarse o enviarse como JSON."""
        return {
            "posicion_pelota": [self.pelota.posicion_x, self.pelota.posicion_y],
            "paleta_izquierda": self.paletas[0].posicion_y,
            "paleta_derecha": self.paletas[1].posicion_y,
            "puntos_izquierda": self.puntos[0],
```

```
            "puntos_derecha": self.puntos[1],
            "ganador": self.ganador,
            "en_pausa": self.en_pausa,
        }
```

A-2. servidor.py

```
"""Servidor de Pong: recibe controles, actualiza la partida y comparte su estado."""

import argparse
from dataclasses import dataclass
import json
from pathlib import Path
import socket
import subprocess
import sys
import time

from modelo import Partida



PUERTO_PREDETERMINADO = 12345
TIEMPO_DESCONEXION = 5.0
VIGENCIA_CONTROLES = 0.3
ACTUALIZACIONES_POR_SEGUNDO = 60
MAXIMO_MENSAJES_POR_CICLO = 256



@dataclass
class JugadorConectado:
    """Asocia una dirección de red con los últimos controles recibidos."""

    direccion: tuple
    ultima_actividad: float
    subir: bool = False
    bajar: bool = False
    ultima_solicitud_pausa: int = 0

    def obtener_direccion(self, instante: float) -> int:
        # Si dejan de llegar controles, detener la paleta evita movimientos fantasma.
        if instante - self.ultima_actividad >= VIGENCIA_CONTROLES:
            return 0
        return int(self.bajar) - int(self.subir)



class ServidorPong:
    """Coordina dos jugadores y conserva la autoridad sobre las reglas del juego."""

    def __init__(self, direccion="0.0.0.0", puerto=PUERTO_PREDETERMINADO):
        self.canal = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        try:
            self.canal.bind((direccion, puerto))
            self.canal.setblocking(False)
        except OSError:
            self.canal.close()
            raise
        self.jugadores = [None, None]
        self.partida = Partida()
        self.secuencia = 0

    def buscar_jugador(self, direccion: tuple):
        for numero, jugador in enumerate(self.jugadores):
            if jugador is not None and jugador.direccion == direccion:
                return numero
        return None

    def retirar_jugador(self, numero: int) -> None:
        self.jugadores[numero] = None
        self.partida.reiniciar()
        print(f"[SERVIDOR] Jugador {numero + 1} desconectado.", flush=True)

    def retirar_inactivos(self, instante: float) -> None:
        for numero, jugador in enumerate(self.jugadores):
            if jugador is not None and instante - jugador.ultima_actividad > TIEMPO_DESCONEXION:
```

```
            self.retirar_jugador(numero)

    def procesar_mensaje(self, mensaje: dict, direccion: tuple, instante: float) -> None:
        """Registra jugadores y aplica sus controles; ignora mensajes desconocidos."""
        if not isinstance(mensaje, dict) or mensaje.get("tipo") not in ("entrada", "salir"):
            return
        numero = self.buscar_jugador(direccion)
        if mensaje["tipo"] == "salir":
            if numero is not None:
                self.retirar_jugador(numero)
            return

        if numero is None:
            numero = next((indice for indice, jugador in enumerate(self.jugadores)
                           if jugador is None), None)
            if numero is None:
                self.enviar({"tipo": "lleno"}, direccion)
                return
            self.jugadores[numero] = JugadorConectado(direccion, instante)
            print(f"[SERVIDOR] Jugador {numero + 1}: {direccion}", flush=True)

        jugador = self.jugadores[numero]
        jugador.ultima_actividad = instante
        jugador.subir = mensaje.get("subir") is True
        jugador.bajar = mensaje.get("bajar") is True
        solicitud = mensaje.get("solicitud_pausa", 0)
        if type(solicitud) is int and solicitud > jugador.ultima_solicitud_pausa:
            pulsaciones = solicitud - jugador.ultima_solicitud_pausa
            jugador.ultima_solicitud_pausa = solicitud
            # El contador evita repetir una pulsación si UDP duplica un paquete.
            # Si se pierden paquetes, se conserva la cantidad de pulsaciones.
            if (pulsaciones % 2 == 1 and all(self.jugadores)
                    and self.partida.ganador is None):
                self.partida.en_pausa = not self.partida.en_pausa
        if self.partida.ganador is not None and mensaje.get("reiniciar") is True:
            self.partida.reiniciar()

    def recibir(self, instante: float) -> None:
        self.retirar_inactivos(instante)
        # El límite permite seguir actualizando el juego aunque lleguen muchos paquetes.
        for _ in range(MAXIMO_MENSAJES_POR_CICLO):
            try:
                datos, direccion = self.canal.recvfrom(2048)
            except (BlockingIOError, ConnectionResetError):
                break
            try:
                mensaje = json.loads(datos.decode("utf-8"))
            except (ValueError, UnicodeError):
                continue
            self.procesar_mensaje(mensaje, direccion, instante)

    def actualizar(self, segundos: float, instante: float) -> None:
        if all(jugador is not None for jugador in self.jugadores):
            direcciones = tuple(jugador.obtener_direccion(instante) for jugador in self.jugadores)
            self.partida.actualizar(segundos, direcciones)

    def enviar(self, mensaje: dict, direccion: tuple) -> None:
        try:
            self.canal.sendto(json.dumps(mensaje).encode("utf-8"), direccion)
        except OSError:
            # UDP no confirma la entrega; la falta de actividad libera el lugar después.
            pass

    def difundir_estado(self) -> None:
        self.secuencia += 1
        estado = self.partida.obtener_estado()
        estado.update(
            tipo="estado", secuencia=self.secuencia,
            jugadores_conectados=sum(jugador is not None for jugador in self.jugadores),
        )
        for numero, jugador in enumerate(self.jugadores):
            if jugador is not None:
                self.enviar(dict(estado, numero_jugador=numero), jugador.direccion)

    def ejecutar(self) -> None:
        """Repite recepción, física y envío unas 60 veces por segundo."""
        instante_anterior = time.monotonic()
        while True:
            inicio = time.monotonic()
            segundos = min(inicio - instante_anterior, 0.05)
            instante_anterior = inicio
```

```
            self.recibir(inicio)
            self.actualizar(segundos, inicio)
            self.difundir_estado()
            duracion = time.monotonic() - inicio
            time.sleep(max(0, 1 / ACTUALIZACIONES_POR_SEGUNDO - duracion))

    def cerrar(self) -> None:
        self.canal.close()



def mostrar_direcciones() -> None:
    """Muestra las IPv4 disponibles; el anfitrión elige la de su red compartida."""
    try:
        resultados = socket.getaddrinfo(socket.gethostname(), None, socket.AF_INET, socket.SOCK_DGRAM)
        direcciones = sorted({resultado[4][0] for resultado in resultados
                              if not resultado[4][0].startswith("127.")})
        if direcciones:
            print("[SERVIDOR] Direcciones de esta PC: " + ", ".join(direcciones), flush=True)
    except OSError:
        pass  # También se pueden consultar manualmente con ipconfig.



def principal() -> None:
    analizador = argparse.ArgumentParser(description=__doc__)
    analizador.add_argument("--direccion", default="0.0.0.0", help="Interfaz de escucha")
    analizador.add_argument("--puerto", type=int, default=PUERTO_PREDETERMINADO)
    analizador.add_argument("--jugar", action="store_true", help="Abrir el cliente local")
    argumentos = analizador.parse_args()
    if not 1 <= argumentos.puerto <= 65535:
        analizador.error("El puerto debe estar entre 1 y 65535")
    try:
        servidor = ServidorPong(argumentos.direccion, argumentos.puerto)
    except OSError as error:
        analizador.exit(1, f"No se pudo iniciar el servidor: {error}\n")

    print(f"[SERVIDOR] Escuchando en {argumentos.direccion}:{argumentos.puerto} (UDP).", flush=True)
    mostrar_direcciones()
    print("El otro jugador debe introducir la IPv4 de esta PC (ver ipconfig).", flush=True)
    proceso_cliente = None
    try:
        if argumentos.jugar:
            direccion_local = "127.0.0.1" if argumentos.direccion == "0.0.0.0" else argumentos.direccion
            ruta_cliente = Path(__file__).with_name("cliente.py")
            proceso_cliente = subprocess.Popen([
                sys.executable, str(ruta_cliente), "--direccion", direccion_local,
                "--puerto", str(argumentos.puerto),
            ])
        servidor.ejecutar()
    except KeyboardInterrupt:
        print("\n[SERVIDOR] Cerrado.")
    finally:
        servidor.cerrar()
        if proceso_cliente is not None and proceso_cliente.poll() is None:
            proceso_cliente.terminate()
            proceso_cliente.wait()



if __name__ == "__main__":
    principal()
```

A-3. cliente.py

```
import argparse
import json
import math
import socket
import time



# Estas medidas coinciden con modelo.py. Se conservan aquí para distribuir
# únicamente este archivo al segundo jugador, sin dependencias del proyecto.
ANCHO_PANTALLA = 800
ALTO_PANTALLA = 600
ANCHO_PALETA = 14
ALTO_PALETA = 90
RADIO_PELOTA = 12
```

```
PUERTO_PREDETERMINADO = 12345
TIEMPO_SIN_RESPUESTA = 5.0
FOTOGRAMAS_POR_SEGUNDO = 60


COLOR_FONDO = (16, 23, 42)
COLOR_RED = (51, 65, 85)
COLOR_TEXTO = (255, 255, 255)
COLOR_SECUNDARIO = (200, 163, 184)
COLOR_PELOTA = (249, 115, 22)
COLOR_DESTACADO = (158, 179, 8)


COLOR_PALETA_IZQUIERDA = (200, 179, 8)
COLOR_PALETA_DERECHA = (155, 255, 255)



def validar_estado(estado) -> bool:
    """Rechaza paquetes incompletos o valores que no se pueden dibujar."""
    if not isinstance(estado, dict) or estado.get("tipo") != "estado":
        return False
    posicion = estado.get("posicion_pelota")
    if not isinstance(posicion, list) or len(posicion) != 2:
        return False
    coordenadas = posicion + [estado.get("paleta_izquierda"), estado.get("paleta_derecha")]
    for valor in coordenadas:
        if type(valor) not in (int, float) or not -100000 < valor < 100000:
            return False
        if not math.isfinite(valor):
            return False
    for nombre in ("puntos_izquierda", "puntos_derecha"):
        if type(estado.get(nombre)) is not int or not 0 <= estado[nombre] <= 10:
            return False
    return (
        type(estado.get("en_pausa", False)) is bool
        and type(estado.get("secuencia")) is int and estado["secuencia"] >= 0
        and type(estado.get("numero_jugador")) is int and estado["numero_jugador"] in (0, 1)
        and type(estado.get("jugadores_conectados")) is int
        and estado["jugadores_conectados"] in (1, 2)
        and (estado.get("ganador") is None or (
            type(estado["ganador"]) is int and estado["ganador"] in (0, 1)
        ))
    )



class ErrorConexion(Exception):
    """Indica un problema de red que la ventana debe mostrar al usuario."""



class ConexionCliente:
    """Envía controles y conserva el estado más reciente recibido del servidor."""

    def __init__(self):
        self.canal = None
        self.estado = None
        self.secuencia = -1
        self.inicio_conexion = 0.0
        self.ultima_recepcion = 0.0
        self.solicitud_pausa = 0

    @property
    def conectada(self) -> bool:
        """Indica si existe un canal abierto; la respuesta del servidor llega después."""
        return self.canal is not None

    def conectar(self, direccion: str, puerto: int) -> None:
        if not direccion.strip() or not 1 <= puerto <= 65535:
            raise ValueError("Introduce una dirección y un puerto entre 1 y 65535.")
        direccion_ip = socket.gethostbyname(direccion.strip())
        self.desconectar()
        canal_nuevo = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        try:
            # En UDP esto selecciona el destino y filtra respuestas de otras direcciones.
            # No establece una sesión como TCP: el estado recibido confirma al servidor.
            canal_nuevo.connect((direccion_ip, puerto))
            canal_nuevo.setblocking(False)
        except OSError:
            canal_nuevo.close()
            raise
        self.canal = canal_nuevo
        self.estado = None
```

```
        self.secuencia = -1
        self.ultima_recepcion = 0.0
        self.inicio_conexion = time.monotonic()
        self.solicitud_pausa = 0

    def solicitar_pausa(self) -> None:
        """Registra una pulsación de P y la reenvía hasta que llegue al servidor."""
        if self.conectada:
            self.solicitud_pausa += 1

    def intercambiar(self, subir: bool, bajar: bool, reiniciar: bool, instante: float) -> None:
        """Envía una entrada y lee las respuestas pendientes sin detener el dibujo."""
        if not self.conectada:
            return
        entrada = {"tipo": "entrada", "subir": subir, "bajar": bajar, "reiniciar": reiniciar}
        entrada["solicitud_pausa"] = self.solicitud_pausa
        try:
            self.canal.send(json.dumps(entrada).encode("utf-8"))
            for _ in range(256):
                try:
                    mensaje = json.loads(self.canal.recv(4096).decode("utf-8"))
                except (ValueError, UnicodeError):
                    continue
                if isinstance(mensaje, dict) and mensaje.get("tipo") == "lleno":
                    raise ErrorConexion("Servidor lleno: ya hay dos jugadores.")
                if validar_estado(mensaje) and mensaje["secuencia"] > self.secuencia:
                    # UDP puede entregar paquetes fuera de orden; descartar los antiguos.
                    self.estado = mensaje
                    self.secuencia = mensaje["secuencia"]
                    self.ultima_recepcion = instante
        except (BlockingIOError, ConnectionResetError, ConnectionRefusedError):
            pass  # Mantener la ventana activa mientras se espera una respuesta.
        except OSError as error:
            raise ErrorConexion(f"Error de red: {error}") from error

        ultima_actividad = self.ultima_recepcion or self.inicio_conexion
        if instante - ultima_actividad > TIEMPO_SIN_RESPUESTA:
            raise ErrorConexion("Sin respuesta. Revisa IP, puerto, red y cortafuegos.")

    def desconectar(self) -> None:
        if self.canal is not None:
            try:
                self.canal.send(json.dumps({"tipo": "salir"}).encode("utf-8"))
            except OSError:
                pass  # Si el aviso se pierde, el servidor detecta la inactividad.
            self.canal.close()
        self.canal = None
        self.estado = None



class PantallaPong:
    """Dibuja la pista y los textos. No conoce sockets ni modifica la partida."""

    def __init__(self, titulo="Pong - Multijugador"):
        # Importar aquí permite probar la red en equipos que no tienen Pygame.
        import pygame

        self.motor = pygame
        self.motor.init()
        self.superficie = self.motor.display.set_mode((ANCHO_PANTALLA, ALTO_PANTALLA))
        self.motor.display.set_caption(titulo)
        self.reloj = self.motor.time.Clock()
        # La fuente incluida en Pygame está disponible tanto en Windows como en Linux.
        self.fuente_grande = self.motor.font.Font(None, 42)
        self.fuente_pequena = self.motor.font.Font(None, 26)

    def limpiar(self) -> None:
        self.superficie.fill(COLOR_FONDO)

    def dibujar_texto(self, texto: str, posicion_y: int, color=COLOR_TEXTO, grande=False) -> None:
        fuente = self.fuente_grande if grande else self.fuente_pequena
        imagen = fuente.render(texto, True, color)
        posicion_x = (ANCHO_PANTALLA - imagen.get_width()) // 2
        self.superficie.blit(imagen, (posicion_x, posicion_y))

    def dibujar_pista(self, estado: dict, numero_jugador=None) -> None:
        for posicion_y in range(0, ALTO_PANTALLA, 30):
            self.motor.draw.rect(self.superficie, COLOR_RED, (398, posicion_y, 4, 15))
        posiciones = (estado["paleta_izquierda"], estado["paleta_derecha"])
```

```
        for lado, posicion_x in enumerate((25, 761)):
            color = COLOR_PALETA_IZQUIERDA if lado == 0 else COLOR_PALETA_DERECHA
            self.motor.draw.rect(
                self.superficie, color,
                (posicion_x, posiciones[lado], ANCHO_PALETA, ALTO_PALETA), border_radius=4,
            )
        centro = tuple(int(coordenada) for coordenada in estado["posicion_pelota"])
        self.motor.draw.circle(self.superficie, COLOR_PELOTA, centro, RADIO_PELOTA)
        marcador = f"{estado['puntos_izquierda']}      {estado['puntos_derecha']}"
        self.dibujar_texto(marcador, 25, grande=True)


    def presentar(self) -> float:
        """Actualiza la ventana y devuelve los segundos transcurridos entre cuadros."""
        self.motor.display.flip()
        return self.reloj.tick(FOTOGRAMAS_POR_SEGUNDO) / 1000


    def cerrar(self) -> None:
        self.motor.quit()



class ClientePong:
    """Relaciona el formulario, las teclas, la conexión y la presentación del juego."""


    def __init__(self, direccion="", puerto=PUERTO_PREDETERMINADO):
        self.pantalla = PantallaPong()
        self.conexion = ConexionCliente()
        self.direccion_escrita = direccion
        self.puerto_escrito = str(puerto)
        self.campo_activo = 0
        self.mensaje_error = ""
        self.en_ejecucion = True
        if direccion:
            self.conectar()


    def conectar(self) -> None:
        try:
            self.conexion.conectar(self.direccion_escrita, int(self.puerto_escrito))
        except (ValueError, OSError) as error:
            self.mensaje_error = str(error)
        else:
            self.mensaje_error = ""


    def editar_formulario(self, evento) -> None:
        motor = self.pantalla.motor
        if evento.key == motor.K_RETURN:
            self.conectar()
        elif evento.key == motor.K_TAB:
            self.campo_activo = 1 - self.campo_activo
        elif evento.key == motor.K_BACKSPACE:
            if self.campo_activo == 0:
                self.direccion_escrita = self.direccion_escrita[:-1]
            else:
                self.puerto_escrito = self.puerto_escrito[:-1]
        elif evento.unicode.isprintable():
            if self.campo_activo == 0:
                self.direccion_escrita = (self.direccion_escrita + evento.unicode)[:100]
            elif evento.unicode.isdecimal():
                self.puerto_escrito = (self.puerto_escrito + evento.unicode)[:5]


    def procesar_eventos(self) -> bool:
        """Devuelve la solicitud de reinicio de este cuadro."""
        reiniciar = False
        motor = self.pantalla.motor
        for evento in motor.event.get():
            if evento.type == motor.QUIT:
                self.en_ejecucion = False
            elif evento.type == motor.KEYDOWN:
                if not self.conexion.conectada:
                    self.editar_formulario(evento)
                elif evento.key == motor.K_ESCAPE:
                    self.conexion.desconectar()
                elif evento.key == motor.K_r:
                    reiniciar = True
                elif evento.key == motor.K_p and not getattr(evento, "repeat", False):
                    self.conexion.solicitar_pausa()
        return reiniciar


    def actualizar_conexion(self, reiniciar: bool, instante: float) -> None:
        motor = self.pantalla.motor
        teclas = motor.key.get_pressed()
```

```
        tiene_foco = motor.key.get_focused()
        subir = bool(tiene_foco and (teclas[motor.K_w] or teclas[motor.K_UP]))
        bajar = bool(tiene_foco and (teclas[motor.K_s] or teclas[motor.K_DOWN]))
        try:
            self.conexion.intercambiar(subir, bajar, reiniciar, instante)
        except ErrorConexion as error:
            self.mensaje_error = str(error)
            self.conexion.desconectar()

    def dibujar_formulario(self) -> None:
        pantalla = self.pantalla
        pantalla.dibujar_texto("PONG", 110, COLOR_PELOTA, grande=True)
        pantalla.dibujar_texto("Conectar al servidor", 185, grande=True)
        campos = ("IP del servidor: " + self.direccion_escrita, "Puerto UDP: " + self.puerto_escrito)
        for numero, texto in enumerate(campos):
            if numero == self.campo_activo:
                texto = "> " + texto + "_"
            pantalla.dibujar_texto(texto, 265 + numero * 45)
        pantalla.dibujar_texto("Tabulador: cambiar campo    Intro: conectar", 385, COLOR_SECUNDARIO)
        pantalla.dibujar_texto(self.mensaje_error[:90], 440, COLOR_DESTACADO)

    def dibujar_partida(self, instante: float) -> None:
        pantalla = self.pantalla
        estado = self.conexion.estado
        numero = estado["numero_jugador"]
        pantalla.dibujar_pista(estado, numero)
        lado = "izquierda" if numero == 0 else "derecha"
        pantalla.dibujar_texto(f"Tu paleta: {lado}", 78, COLOR_DESTACADO)
        if instante - self.conexion.ultima_recepcion > 1:
            pantalla.dibujar_texto("Se perdió la conexión...", 285, COLOR_DESTACADO, grande=True)
        elif estado["jugadores_conectados"] < 2:
            pantalla.dibujar_texto("Esperando al otro jugador... (1/2)", 285, COLOR_DESTACADO, grande=True)
        elif estado["ganador"] is not None:
            mensaje = "¡Ganaste!" if estado["ganador"] == numero else "Ganó tu oponente"
            pantalla.dibujar_texto(mensaje, 270, COLOR_DESTACADO, grande=True)
            pantalla.dibujar_texto("R: nueva partida", 320)
        elif estado.get("en_pausa", False):
            pantalla.dibujar_texto("JUEGO PAUSADO", 270, COLOR_DESTACADO, grande=True)
            pantalla.dibujar_texto("P: reanudar para ambos jugadores", 320)
        pantalla.dibujar_texto("W/S o flechas: mover  |  P: pausa  |  Esc: desconectar", 560, COLOR_SECUNDARIO)

    def ejecutar(self) -> None:
        """Orden del ciclo: eventos, intercambio de red, dibujo y espera del reloj."""
        try:
            while self.en_ejecucion:
                reiniciar = self.procesar_eventos()
                if not self.en_ejecucion:
                    break
                instante = time.monotonic()
                self.actualizar_conexion(reiniciar, instante)
                self.pantalla.limpiar()
                if not self.conexion.conectada:
                    self.dibujar_formulario()
                elif self.conexion.estado is not None:
                    self.dibujar_partida(instante)
                else:
                    self.pantalla.dibujar_texto("Conectando al servidor...", 275, COLOR_DESTACADO, grande=True)
                    self.pantalla.dibujar_texto("Esc: volver", 340, COLOR_SECUNDARIO)
                self.pantalla.presentar()
        finally:
            self.conexion.desconectar()
            self.pantalla.cerrar()



def principal() -> None:
    analizador = argparse.ArgumentParser(description=__doc__)
    analizador.add_argument("--direccion", default="", help="IP o nombre del servidor")
    analizador.add_argument("--puerto", type=int, default=PUERTO_PREDETERMINADO)
    argumentos = analizador.parse_args()
    try:
        cliente = ClientePong(argumentos.direccion, argumentos.puerto)
    except ModuleNotFoundError as error:
        if error.name != "pygame":
            raise
        analizador.exit(1, "Falta Pygame. Instálalo con: python -m pip install pygame\n")
    cliente.ejecutar()



if __name__ == "__main__":
    principal()
```

A-4. juego.py

```
"""Modo local: dos personas juegan desde el teclado de una misma computadora."""

from cliente import COLOR_DESTACADO, COLOR_SECUNDARIO, PantallaPong
from modelo import Partida



class JuegoLocal:
    """Compone una partida y una pantalla; administra los controles del teclado."""

    def __init__(self):
        self.partida = Partida()
        self.pantalla = PantallaPong("Pong - Dos jugadores locales")
        self.en_ejecucion = True
        self.en_pausa = False

    def procesar_eventos(self) -> None:
        motor = self.pantalla.motor
        for evento in motor.event.get():
            if evento.type == motor.QUIT:
                self.en_ejecucion = False
            elif evento.type == motor.KEYDOWN:
                if evento.key == motor.K_r or self.partida.ganador is not None:
                    self.partida.reiniciar()
                    self.en_pausa = False
                elif evento.key == motor.K_p:
                    self.en_pausa = not self.en_pausa

    def actualizar(self, segundos: float) -> None:
        if self.en_pausa:
            return
        motor = self.pantalla.motor
        teclas = motor.key.get_pressed()
        izquierda = int(teclas[motor.K_s]) - int(teclas[motor.K_w])
        derecha = int(teclas[motor.K_DOWN]) - int(teclas[motor.K_UP])
        self.partida.actualizar(segundos, (izquierda, derecha))

    def dibujar(self) -> None:
        self.pantalla.limpiar()
        self.pantalla.dibujar_pista(self.partida.obtener_estado())
        self.pantalla.dibujar_texto("Izquierda: W/S    Derecha: flechas    P: pausa    R: reiniciar", 560,
COLOR_SECUNDARIO)
        if self.partida.ganador is not None:
            lado = "izquierdo" if self.partida.ganador == 0 else "derecho"
            self.pantalla.dibujar_texto(f"¡Gana el jugador {lado}!", 270, COLOR_DESTACADO, grande=True)
            self.pantalla.dibujar_texto("Presiona cualquier tecla para reiniciar", 320)
        elif self.en_pausa:
            self.pantalla.dibujar_texto("JUEGO PAUSADO", 275, grande=True)

    def ejecutar(self) -> None:
        segundos = 0.0
        try:
            while self.en_ejecucion:
                self.procesar_eventos()
                if not self.en_ejecucion:
                    break
                # Limitar el salto temporal evita avances grandes si se arrastra la ventana.
                self.actualizar(min(segundos, 0.05))
                self.dibujar()
                segundos = self.pantalla.presentar()
        finally:
            self.pantalla.cerrar()



def principal() -> None:
    juego = JuegoLocal()
    juego.ejecutar()



if __name__ == "__main__":
    principal()
```